Recover the Design

Author: YiFeng Zheng

Reviewer: YuJie Sun

Group number: 17

Date: 2026-10-23

Abstract

We recover a fence layout from the degrees of the connectors on a grid. We first turn the grid into a graph: two connectors are joined by a candidate edge when a straight fence can link them. Then we reject inputs whose degree sums can never work, and write the remaining rules as Boolean clauses. A small SAT solver written in C++17 searches for a choice of edges, learns from every conflict and backtracks. The program passes 1,170 grid tests and 2,000 Boolean tests with both a release build and a sanitizer build. On generated 50 by 50 inputs it finishes in at most about 0.2 seconds, but the worst case is still exponential.

Contents

1 Introduction **3**

2 Data Structure / Algorithm Specification **3**

2.1 Candidate Edges 3

2.2 Quick Checks on Degree Sums 4

2.3 Writing the Rules as Clauses 4

2.4 Search with Learning and Backtracking 4

2.5 Correctness Argument 5

3 Testing Results **6**

3.1 Environment and Method 6

3.2 Correctness Tests 6

3.3 Timing Results 8

4 Analysis and Comments **9**

4.1 Time Complexity 9

4.2 Space Complexity 10

4.3 Discussion of Test Results 10

4.4 Limitations and Improvements 10

5 Conclusion **11**

6 References **11**

7 Author List **11**

8 Declaration **11**

9 Signatures **11**

10 Source Code Appendix **11**

1 Introduction

The PTA problem "Recover the Design" gives an $n \times m$ grid with $1 < n, m \leq 50$. Each cell holds 0 (no connector) or a number k from 1 to 4, which says that this connector must be linked by fences in exactly k of its four directions. Fences only run horizontally or vertically. A fence can not bend in an empty cell, and two fences can not cross in an empty cell; turns and crossings are only allowed at connectors. When a fence goes straight through a connector, it uses two directions of that connector. The connectors do not need to form one connected piece.

For every connector, in row order and from left to right, we print `row col up down left right`. Coordinates start at 1 and each direction is 0 or 1. Numbers are separated by one space with no extra spaces at the ends. If no layout exists we print `No Solution`. Any valid layout is accepted, and a grid without connectors gives empty output.

The task is hard because the choices depend on each other. A dense 50 by 50 grid has up to 2,500 connectors and almost 5,000 places where a fence could be built, so trying every subset (2^{5000} cases) is impossible. A choice made at one connector changes the degrees of its neighbours, and a long fence can block a fence in the other direction far away. So we need a model that states all rules clearly, quick checks that throw away hopeless inputs early, and a search that does not repeat the same mistake.

2 Data Structure / Algorithm Specification

2.1 Candidate Edges

We model the layout as a graph. Each connector is a vertex. If two connectors are in the same row or column and only empty cells lie between them, we join them with a candidate edge. Each vertex has at most four candidate edges, one per direction. A long fence that goes through several connectors is just a chain of these edges, because the fence uses two directions of every connector it passes. So we never need an edge that skips over a connector.

Let $x_e \in \{0, 1\}$ say whether edge e is built. For a vertex v with degree k_v we need $\sum_{e \ni v} x_e = k_v$. If a horizontal edge e and a vertical edge f pass the same empty cell, they cross there, so we need $x_e + x_f \leq 1$.

To build the graph, we scan every row from left to right and join each connector with the next one, and then do the same for every column. While adding an edge we also write its id into the empty cells it passes, using one grid for horizontal edges and one for vertical edges. A cell that has both a horizontal and a vertical edge gives one crossing pair.

The small grid below shows a crossing. Edge h joins the two connectors in the middle row and edge v joins the two connectors in the middle column. Both pass the empty center cell, so at most one

of them can be built. Every connector has degree 1 and only one candidate edge, so both edges are required, and the answer is `No Solution`.

```
. 1 .         . | .
1 . 1         - + -     h = middle row, v = middle column
. 1 .         . | .     they meet at the empty center cell
```

2.2 Quick Checks on Degree Sums

Before searching, we check each connected part of the candidate graph. Every built edge adds 1 to the degree of both of its ends, so the degree sum of a part must be even. We also color each part with two colors by breadth-first search. If this works (the part is bipartite), every edge has one end of each color, so the two colors must have the same degree sum. If a part has an odd cycle, only the even-sum check is used.

We color the real candidate graph instead of using the grid parity $(r + c) \bmod 2$. The reason is that a long edge can join two cells with the same parity, for example the cells $(1, 1)$ and $(1, 3)$ when $(1, 2)$ is empty. These checks cost linear time and reject many impossible inputs before any search, for example a grid of all 1s with an odd number of cells.

2.3 Writing the Rules as Clauses

Each candidate edge is one Boolean variable. A clause is a list of conditions, and at least one of them must hold. A connector with d candidate edges and degree k is rejected at once if $k > d$. Otherwise we write two kinds of clauses:

1. Any $k + 1$ of its edges can not all be built, so for every such subset at least one edge is not built.
2. Any $d - k + 1$ of its edges can not all be unused, so for every such subset at least one edge is built.

Together they mean exactly k edges are built. For example, with $d = 3$ edges a, b, c and $k = 2$, we get $(\neg a \vee \neg b \vee \neg c)$, $(a \vee b)$, $(a \vee c)$ and $(b \vee c)$. Since $d \leq 4$, each connector gives at most a few short clauses. Each crossing pair adds the clause $(\neg x_e \vee \neg x_f)$.

2.4 Search with Learning and Backtracking

We wrote a small SAT solver in the same source file; no outside library is used. It follows the standard conflict-driven clause learning (CDCL) method [1]. The main loop is:

```
solve():
    loop:
        propagate()                      # set every value forced by a clause
        if some clause is false:
            if no decision has been made: return NO SOLUTION
            learn a new clause from the conflict
            jump back to an earlier decision level
            set the literal forced by the new clause
            restart from level 0 after enough conflicts (keep learned clauses)
        else if every variable has a value:
            return the values             # a valid layout
        else:
            pick the free variable with the highest score
            try the value it had last time (first time: build the edge)
```

Propagation. Each clause watches two of its literals. When a watched literal becomes false, we look for another literal in the clause that is not false and watch it instead. If there is none, the other watched literal must become true. If that one is false too, we have a conflict. In this way we only look at the clauses that may have changed, not the whole graph.

Learning. For every forced value we store the clause that forced it (its reason) and the decision level. After a conflict, we start from the false clause and replace literals of the current level by their reasons, until only one literal of the current level is left. This gives a new clause that is a logical result of the old ones. We keep it, so the search will never try the same bad combination again.

Backtracking and restarts. The new clause tells us how far to go back: to the highest level among its other literals, which may skip several levels at once. There the new clause forces one value, and the search goes on. Variables that appear in conflicts get a higher score, so they are decided earlier next time. After 100 conflicts we restart from level 0 and keep all learned clauses; the limit grows by 1.5 times each restart, up to 10,000. The solver has no time limit, so it never reports `No Solution` just because it ran for a long time.

2.5 Correctness Argument

The model is complete: any legal layout can be cut at its connectors into candidate edges, and those edges satisfy all degree clauses and crossing clauses. The model is also sound: the degree clauses allow exactly k built edges at each connector, and the crossing clauses forbid exactly the crossings in empty cells. Since fences only exist on candidate edges, they are straight and only turn at connectors.

The quick checks only reject an input when a degree sum shows that no edge set can match the degrees, so they never reject a solvable input. Propagation only sets a value that a clause forces. A learned clause is obtained by resolution of existing clauses, so every valid layout still satisfies it

[1]. Backtracking removes values but never removes clauses. Since there are finitely many variables and learned clauses are never repeated, the CDCL search ends [1]; the general argument is in the reference, and our exhaustive tests check this implementation separately.

If the search ends with every variable set, all clauses hold, and we print each built edge at both of its ends in input order. If a conflict appears before any decision, the clauses can not be satisfied, so `No Solution` is correct.

3 Testing Results

3.1 Environment and Method

All tests and timings in this chapter were run on Linux with an Intel Xeon CPU at 2.10 GHz (4 cores), GCC 13.3.0 and Python 3.13. The release build uses `-std=c++17 -O2 -Wall -Wextra -Wpedantic`. A second build turns on AddressSanitizer and UndefinedBehaviorSanitizer to find memory errors and undefined behaviour.

We check outputs with an independent Python checker. It reads every printed line and follows each direction cell by cell. It checks the degree of each connector, that both ends of a fence agree, the output order and spacing, and that no two fences cross in an empty cell. For small grids we also count all subsets of candidate edges by brute force, so we know whether `No Solution` is the right answer.

3.2 Correctness Tests

Besides the grid tests, we test two parts of the program on their own. To test the SAT solver alone, a separate program includes our source file and runs 2,000 random formulas with 1 to 8 variables. Each answer is compared with a full truth table, and each returned assignment is checked against every clause. All 2,000 pass.

To make sure learning and backtracking really run, a script builds a copy of the program with counters added; the search itself is unchanged. On a 4 by 4 grid that needs search and has no solution, the solver meets 2 conflicts, learns 1 clause and backtracks once before it proves `No Solution`; brute force confirms this answer. On a planted 50 by 50 grid with density 1.0, it meets 412 conflicts, learns 412 clauses, jumps back over more than one level 280 times, and restarts twice before it finds a legal layout.

The table below lists the grid tests and what each group is for.

Test group (count)	Result	Purpose
Sample input (1)	PASS	Check the example from the problem gives a legal layout.
All 2 by 2 grids (625)	PASS	Smallest size; every possible degree grid is compared with brute force, including all no-solution cases.
Thin grids (152)	PASS	Minimum width or height: random 2 by m and n by 2 grids ($3 \leq m, n \leq 8$) checked by brute force, plus a 2 by 50 and a 50 by 2 strip.
Random 3 by 3 grids (200)	PASS	Random degrees 0 to 4 with many impossible inputs; compared with brute force.
Hand-made cases (4)	PASS	Crossing in an empty cell must be refused; crossing at a degree 4 connector is allowed; a fence goes straight through a connector; a square with two solutions.
Grids with several solutions (50)	PASS	3 by 4 grids with at least two legal layouts; any one legal output must be accepted.
Planted 4 by 4 grids (100)	PASS	More solvable inputs made from random fences that do not cross.
Special 50 by 50 grids (5)	PASS	Largest size: empty grid (no output), all degree 1, full grid, long row fences, and all degree 4 (no solution).
Degree-sum checks (9)	PASS	Odd total degree (9 by 9, 49 by 49), unequal color sums (10 by 10, 50 by 50), two odd parts with an even total, and solvable long-edge and odd-cycle grids that the checks must not reject.
Planted 20 by 20 and 50 by 50 grids (24)	PASS	Large solvable inputs with densities 0.25, 0.6 and 1.0; each must finish within 2 seconds.

All 1,170 grid cases pass in both builds, and the sanitizer reports no error. We also make sure the checker itself works: it rejects three broken outputs (wrong degree, a fence with only one end, and a crossing in an empty cell).

3.3 Timing Results

Large inputs come from the generator `code/gen.cpp`, which is included in the submission. It has three kinds of input:

1. **planted**: place random straight fences that never cross in an empty cell, then write each connector's degree. These inputs surely have a solution.
2. **broken**: make a planted grid, then add 1 to one connector and take 1 from another. The total degree stays the same, so the quick checks often can not reject it and the solver must search. Some of these inputs still have a solution and some do not.
3. **random**: give each connector a random degree from 1 to 4. Almost all of these have no solution.

We test n by n grids for $n = 10, 20, 30, 40, 50$. For each kind and size we use five seeds. Each input runs once as warmup and then 10 more times; we take the median of the 10 runs, and then the median over the five inputs. The time is measured around the whole process, so it includes about 2 ms for starting the program and reading input. Every printed layout is checked by the checker above; for the large broken and random inputs, the `No Solution` answers are not checked by brute force, since the grids are too large.

Input kind	10	20	30	40	50
planted 1.0	2.6	3.7	9.7	42.0	121.6
planted 0.5	2.5	2.7	3.8	6.1	8.8
broken 1.0	2.7	4.0	8.7	36.5	95.2
random 0.6	2.4	2.5	2.9	2.9	3.1

The table gives the median time in milliseconds for each input kind (with its connector density) and grid size n. The figure shows the same medians on a log scale, and each error bar goes from the fastest to the slowest of the five inputs.

The slowest single input was a solvable broken 50 by 50 grid at 182 ms. Of the 25 broken inputs, 14 had a solution and 11 did not.

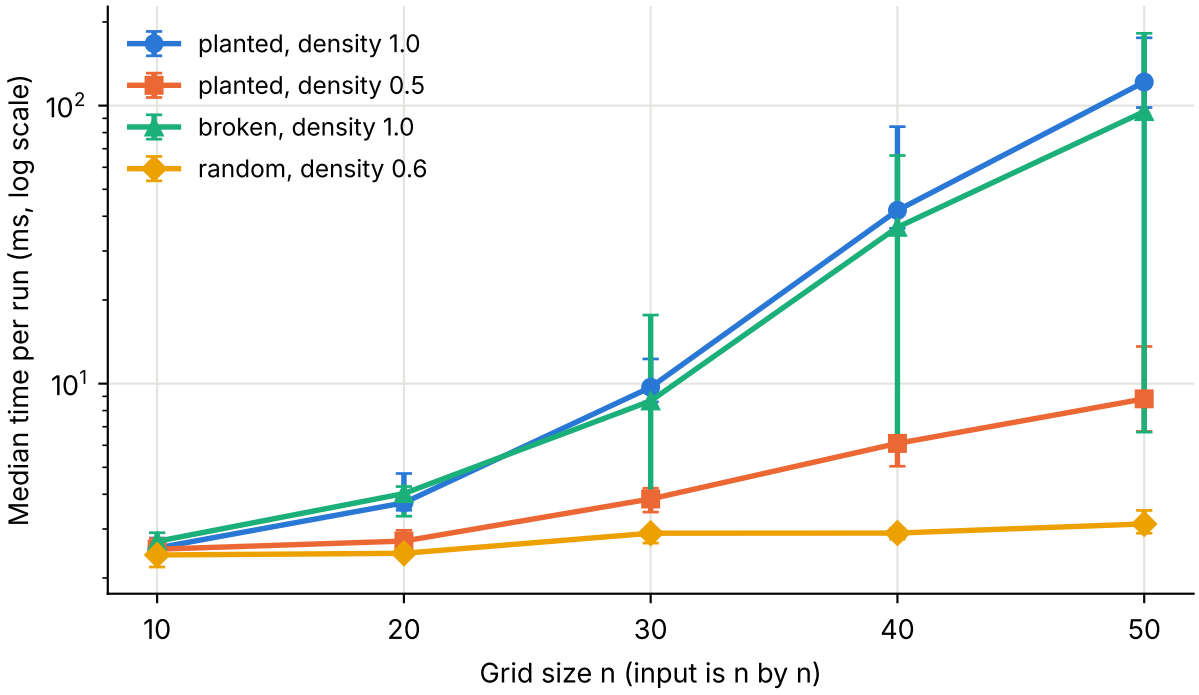


Figure 1: Median running time on generated n by n inputs

4 Analysis and Comments

4.1 Time Complexity

Let V be the number of connectors, E the number of candidate edges and X the number of crossing pairs. Each connector has at most four edges and each edge has two ends, so $E \leq 2V$. Each empty cell gives at most one crossing pair, so $X \leq nm$. Reading the grid, building the edges, running the quick checks and writing the first clauses all take $O(nm + V + E + X)$ time. The degree clauses have constant size per connector because $d \leq 4$.

For the search, let K be the number of conflicts and L the largest total length of all stored clauses, learned clauses included. Between two conflicts at most E variables get a value. Each value can make us look at many watching clauses, so propagation in this period costs at most $O(EL)$. Each decision scans all variables to find the best score, which costs $O(E)$ per decision and $O(E^2)$ per period. Conflict analysis looks at most at E reason clauses of length at most E, so $O(E^2)$. Adding these up, the search costs $O((K + 1)E(E + L))$ on top of building the graph.

This bound comes from our own loops and is loose. More importantly, K is not bounded by a polynomial: in the worst case CDCL can need an exponential number of conflicts, and the learned clauses can grow just as much [1]. So the worst-case time is exponential.

4.2 Space Complexity

The grid, the vertices and the edges use $O(nm + V + E + X)$ space. The clauses and their watch lists use $O(L)$. The values, levels, reasons, scores and the trail of the solver use $O(E)$, and the breadth-first search in the quick checks uses $O(V)$. So the total space is $O(nm + V + E + X + L)$. Since we keep every learned clause, L can also grow exponentially in the worst case. We accept this because keeping the clauses is what stops the search from repeating old mistakes.

4.3 Discussion of Test Results

The timing results match the analysis. To see where the time goes, we also ran the generated inputs with the counter build from the previous section. Random inputs stay near 3 ms even at 50 by 50, which is almost only the start-up time. Their degrees rarely balance, so the linear quick checks reject all of them before the search starts, and $K = 0$.

Planted inputs with density 0.5 grow slowly, to 8.8 ms at 50 by 50. Many connectors have only one or two candidate edges, so propagation fixes most values by itself. The five 50 by 50 inputs needed only 19 to 242 conflicts.

Dense inputs (density 1.0) are the hardest. Every connector has up to four edges and many local choices look fine at first, so the five 50 by 50 inputs needed 589 to 1,526 conflicts. Their time grows much faster than the grid area: from 30 by 30 to 50 by 50 the area grows 2.8 times but the time grows about 12 times. This fits the $(K+1)E(E+L)$ term, since both K and L grow with the grid. Broken inputs behave like planted ones when they still have a solution. When they do not, the time spreads widely: some are refused in about 7 ms, while one 40 by 40 grid needed 369 conflicts and 36.5 ms before the solver proved `No Solution`.

All inputs we tried, including the largest 50 by 50 grids, finish within 0.2 seconds. This shows the search is fast in practice for this problem size, but it is not a proof for every input.

4.4 Limitations and Improvements

1. The worst case is still exponential. Our large tests are generated, and they may not include the hardest 50 by 50 inputs, especially ones with no solution that need a long search. We also have not run the program on the hidden PTA tests.
2. Learned clauses are never deleted, so memory can keep growing on hard inputs. A careful rule that removes old, unused learned clauses would limit memory, as long as clauses that are still reasons are kept.
3. Each decision scans all variables to find the one with the highest score, which costs $O(E)$. Keeping the free variables in a binary max-heap ordered by score would make this $O(\log E)$ per decision.

5 Conclusion

We turned the fence puzzle into a graph of candidate edges and wrote the degree and crossing rules as Boolean clauses. Linear degree-sum checks reject many impossible inputs early. For the rest, a CDCL search with learning and backtracking finds a legal layout or proves there is none. The tests cover the smallest grids by brute force, the special cases from the problem, and generated inputs up to 50 by 50, which all finish within 0.2 seconds. The worst case is still exponential, as the analysis shows.

6 References

[1] N. Eén and N. Sörensson, "An Extensible SAT-solver," in *Theory and Applications of Satisfiability Testing*, LNCS, vol. 2919. Berlin, Heidelberg, Germany: Springer, 2004, pp. 502–518. doi:10.1007/978-3-540-24605-3_37.

7 Author List

YiFeng Zheng: code, tests, report and responses to review comments.

YuJie Sun: independent reproduction and approval.

8 Declaration

We hereby declare that all work in this project, titled "Recover the Design," is our independent effort as a group.

9 Signatures

YiFeng Zheng

YuJie Sun

10 Source Code Appendix

The commented C++17 source is in the `code/` folder of the submission. `code/main.cpp` is the solver; its main parts are `BooleanSolver` (propagation, conflict analysis, backtracking and the search loop) and `Solver` (reading the grid, building candidate edges, quick checks, writing clauses and printing the answer). `code/gen.cpp` is the generator for the large test inputs. The `README.md` in the submission gives the commands to compile and run both programs. The checker and test scripts are kept in our working directory and are not part of the submission.